



Computer Vision: Concepts & Applications

Mentors:

Prasun Shrivastav

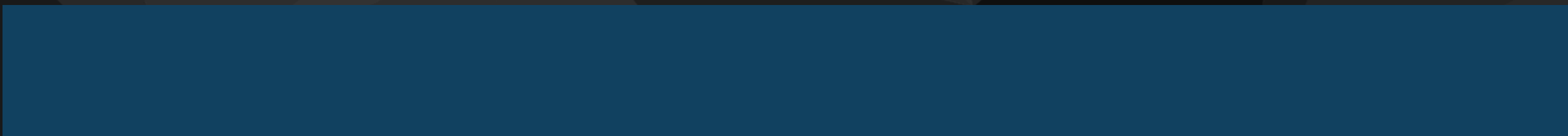
Preet Mehta

Neel Basak

• • • •

• • • •

• • • •



Mentees



Arjun Singla

Arushia

Dev Shakya

Diya Gupta

Harisaran Singh

Kaushik Kumar

Manant Singhal

Pankaj Khoriya

Sarthak Patel

Shailendra Kumar

Shlesha Chawla

Varshini Ridikka S K



WEEK 1

This week, we learnt about :

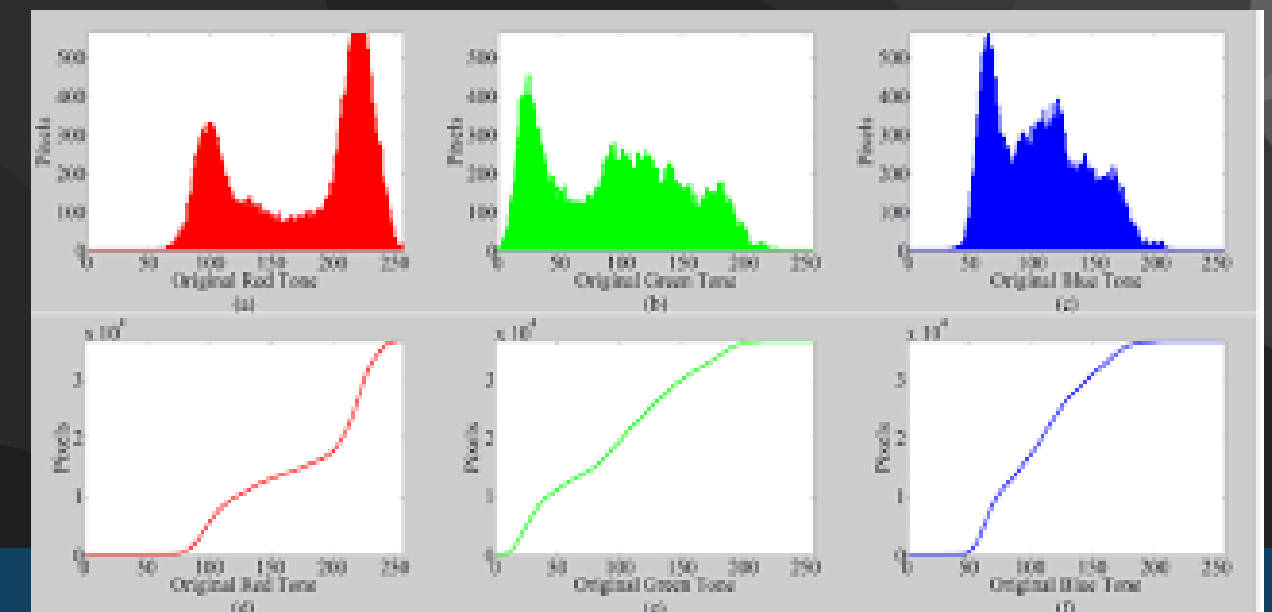
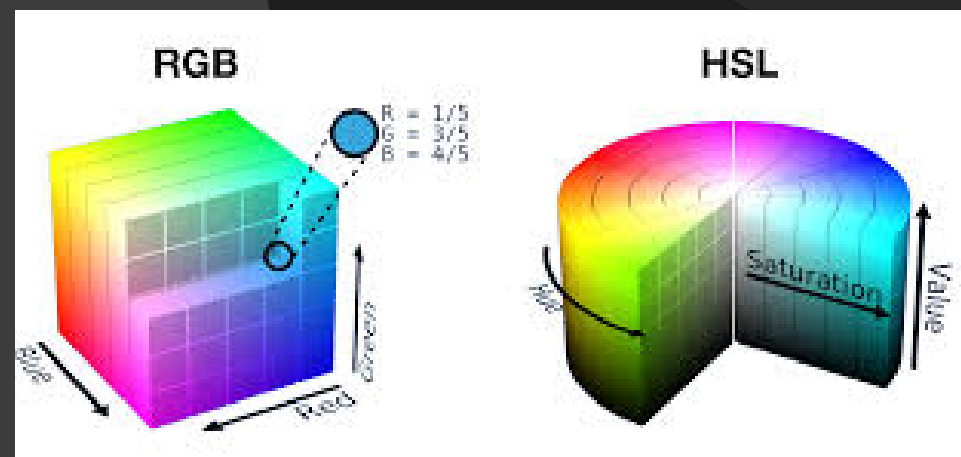
- Color Spaces
- Image Histogram
- Histogram Equalization
- Histogram of Gradients
- Convolution



Color Space

Image Histogram:

- Histograms plot pixel intensity frequencies (0-255), aiding in analyzing and enhancing image contrast.
- Cumulative frequency plots illustrate the proportion of pixels below specific intensity values, guiding adjustments.

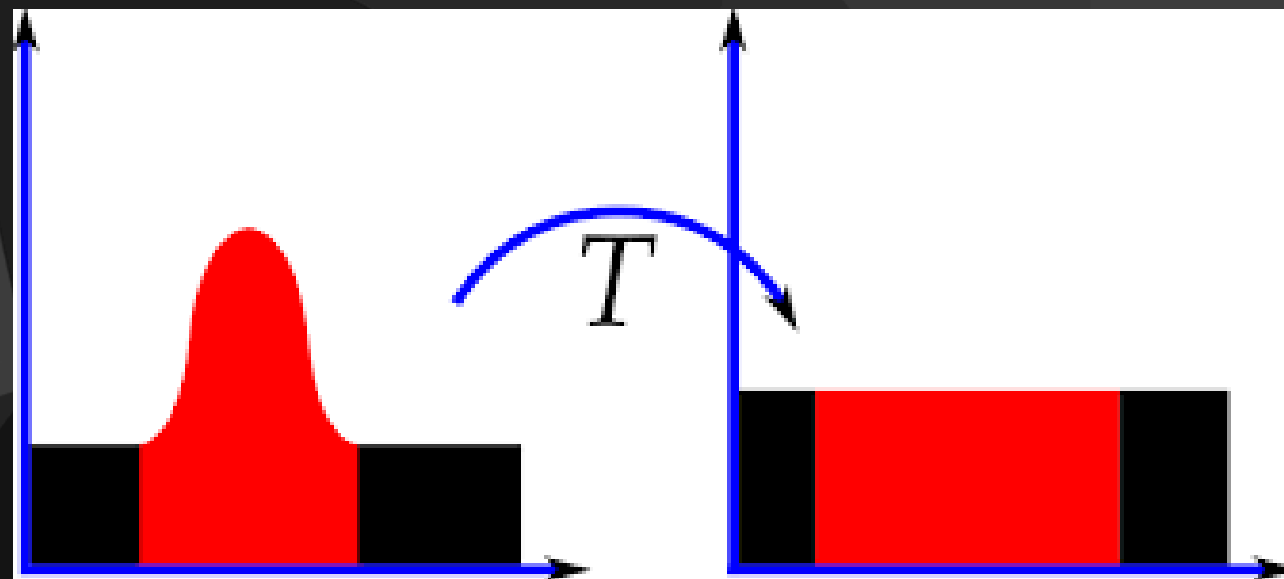


Histogram Equalization:

- **Purpose:** Improves visual quality by enhancing contrast through uniform intensity distribution.
- **Process:** Transforms pixel intensities using cumulative frequency plots, balancing low and high values.
- **Benefits:** Increases image clarity and dynamic range, particularly in darker images.
- **Consideration:** Overuse can degrade image quality; apply carefully.

Histogram of Gradients:

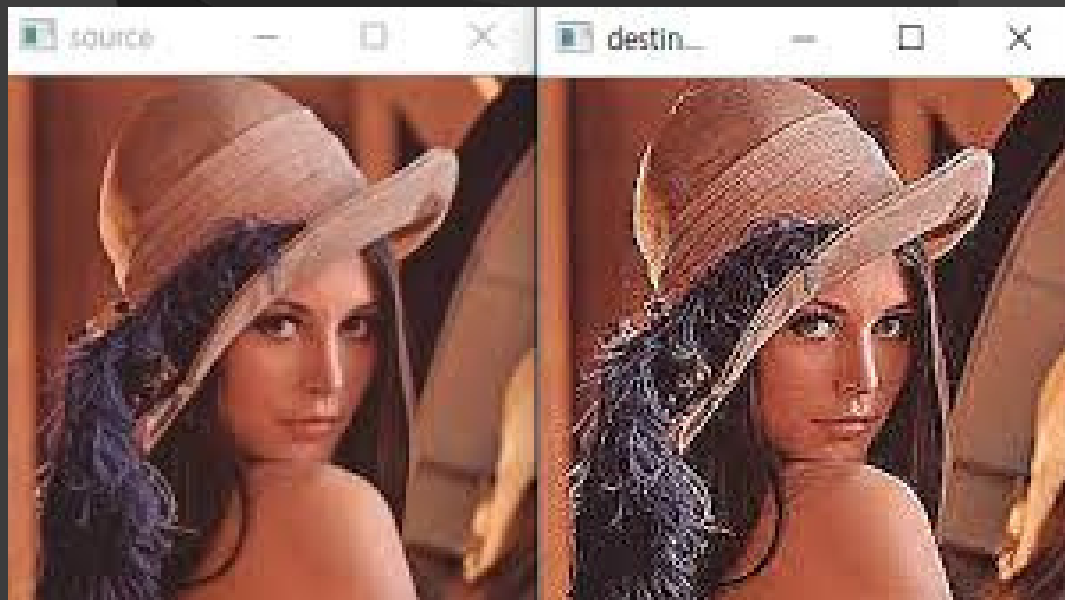
- **Definition:** HOG captures gradient orientations to assist in object and face recognition.
- **Gradient Analysis:** Utilizes directional derivatives to identify edge orientations.
- **Applications:** Improves detection accuracy by focusing on structural features.
- **Settings:** Block sizes (8-16 pixels) optimize edge detection and maintain detail.



CONVOLUTION: BLURRING AND SHARPENING

Convolution is a fundamental image processing operation where a **small matrix called a kernel** is applied to an image to modify pixel values based on their neighbors. By changing the kernel matrix, convolution can be used to either smooth (blur) or enhance edges (sharpen) in an image.

`gaussian_blur = cv2.GaussianBlur(image, (5, 5), 0)`
reduces noise while **preserving edges** better than simple averaging



```
# Creating our 3 x 3 kernel
kernel_3x3 = np.ones((3, 3), np.float32) / 9
# We use the cv2.filter2D to convolve the kernel with an image
blurred = cv2.filter2D(image, -1, kernel_3x3)
imshow('3x3 Box Blurring', blurred)
kernel_sharpening = np.array([[ -1, -1, -1],
                               [ -1,  9, -1],
                               [ -1, -1, -1]])

# applying the sharpening kernel to the image
sharpened = cv2.filter2D(image, -1, kernel_sharpening)
imshow('Sharpened Image', sharpened)
```

WEEK 2

This week, we learnt about :

- Open CV library
- TRANSFORMATIONS
- THRESHOLDING Methods



OpenCV



OpenCV (Open Source Computer Vision Library) is an open-source library used for image and video processing. It provides fast and efficient tools for tasks such as image filtering, color space conversion, geometrical transformations, and thresholding.

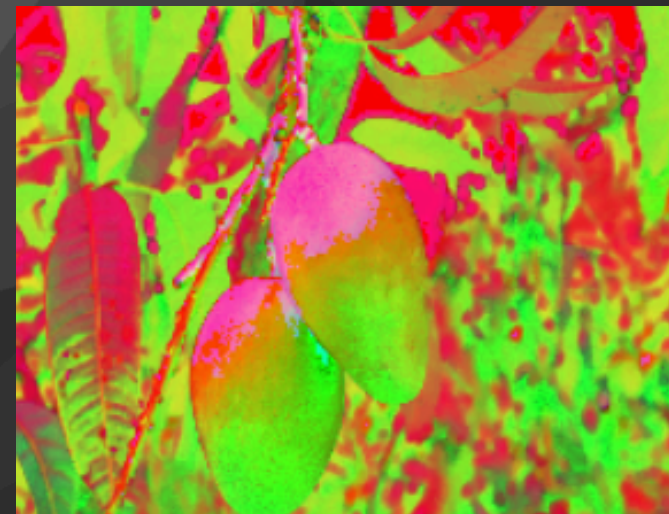
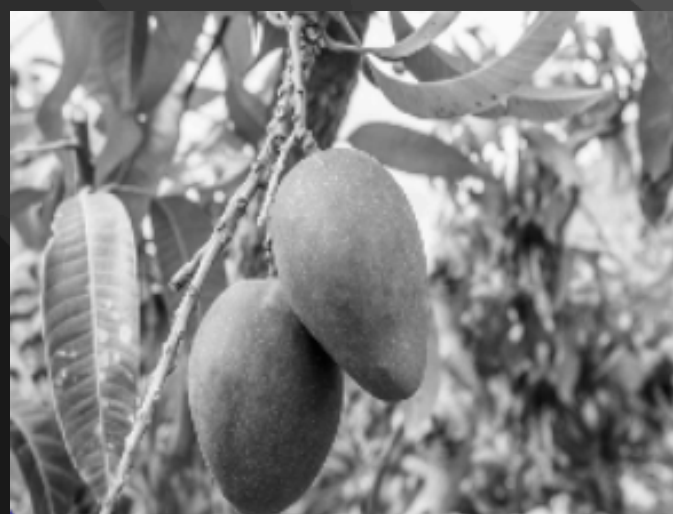


Image loading and display

Load an image using `image = cv2.imread('my_img.jpeg')`

The `imshow()` function is used to display the image in a window. and `waitKey()` is required to keep the window open, and `destroyAllWindows()` closes all image windows.

- OpenCV uses the BGR color space by default.
- `cvtColor()` is used to convert an image from one color space to another
- `gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)`
- `hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)`



Transformations - Translations and Rotations

TRANSLATION

We use `cv2.warpAffine(image, T, (width, height))` to shift image position.
(Left, Right, Up or Down).

$$T = \begin{bmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \end{bmatrix}$$

T_x - Represents the shift along the x-axis

T_y - Represents the shift along the y-axis

ROTATION

```
cv2.getRotationMatrix2D(rotation_center_x, rotation_center_y, angle of rotation, scale)
```

For Eg:-

Divide (h,w) by two to rotate the image around its centre

`rotation_matrix = cv2.getRotationMatrix2D((width/2, height/2), -45, 1)`

Input our image, the rotation matrix and our desired final width and height

`rotated_image = cv2.warpAffine(image, rotation_matrix, (width, height))`

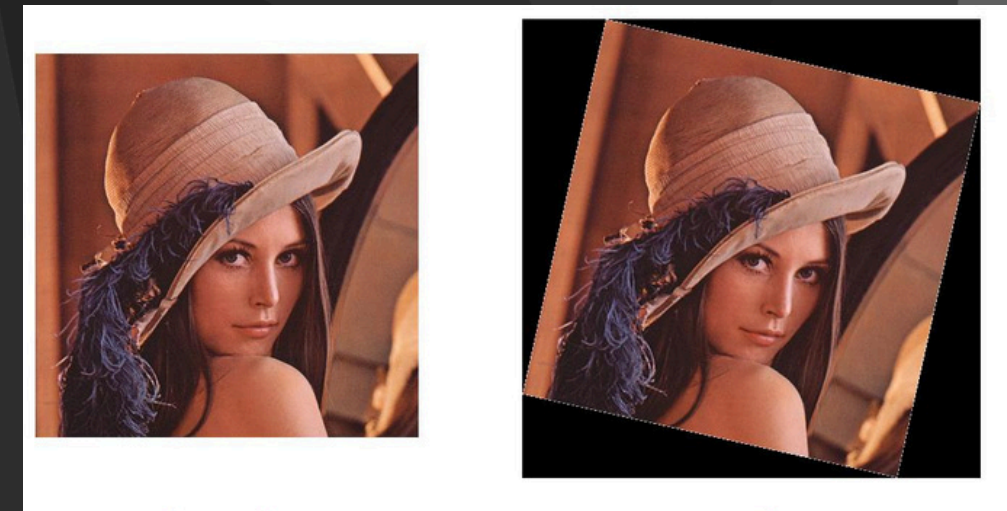


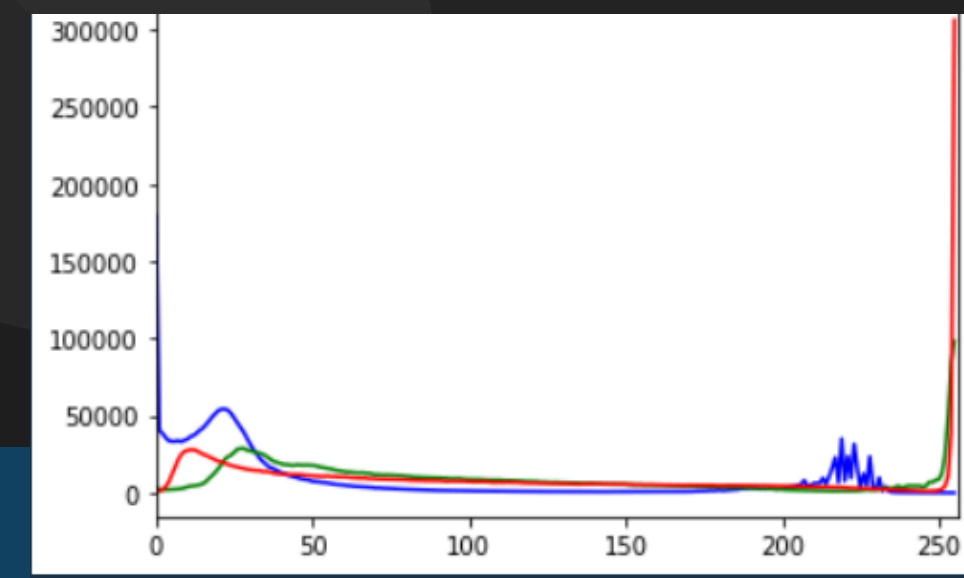
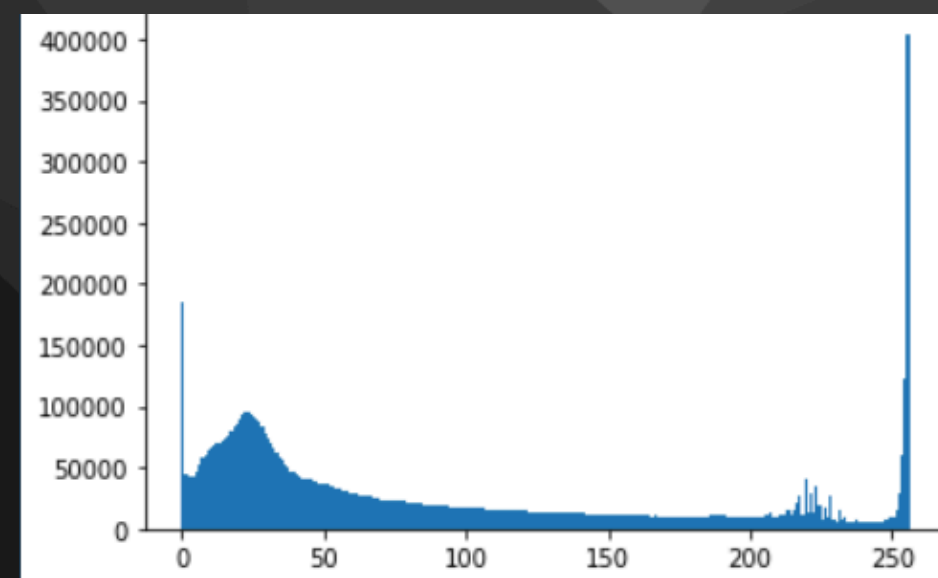
IMAGE HISTOGRAM

An image histogram represents the distribution of pixel intensity values in an image. It shows how many pixels have each possible intensity value, which helps in analyzing contrast, brightness, and intensity variation in an image.

We use the following function to plot histogram using open cv.

```
cv2.calcHist(images, channels, mask, histSize, ranges[, hist[, accumulate]])
```

- **images** : it is the source image of type uint8 or float32. it should be given in square brackets, ie, "[img]".
- **channels** : it is also given in square brackets. It is the index of channel for which we calculate histogram. For example, if input is grayscale image, its value is [0]. For color image, you can pass [0], [1] or [2] to calculate histogram of blue, green or red channel respectively.
- **mask** : mask image. To find histogram of full image, it is given as "None". But if you want to find histogram of particular region of image, you have to create a mask image for that and give it as mask. (I will show an example later.)
- **histSize** : this represents our BIN count. Need to be given in square brackets. For full scale, we pass [256].
- **ranges** : this is our RANGE. Normally, it is [0,256].

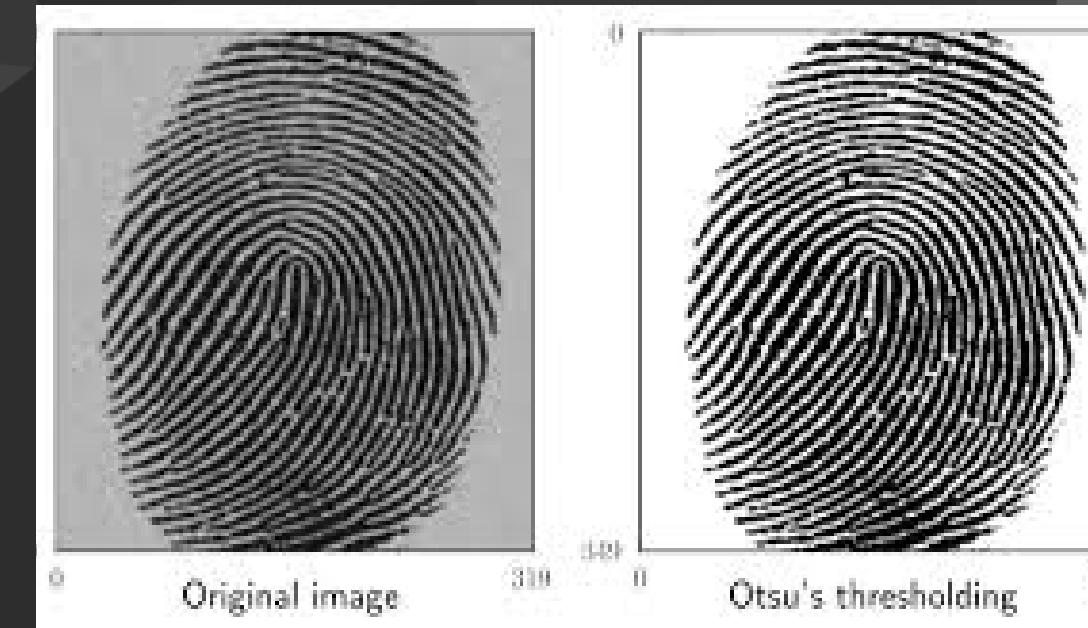
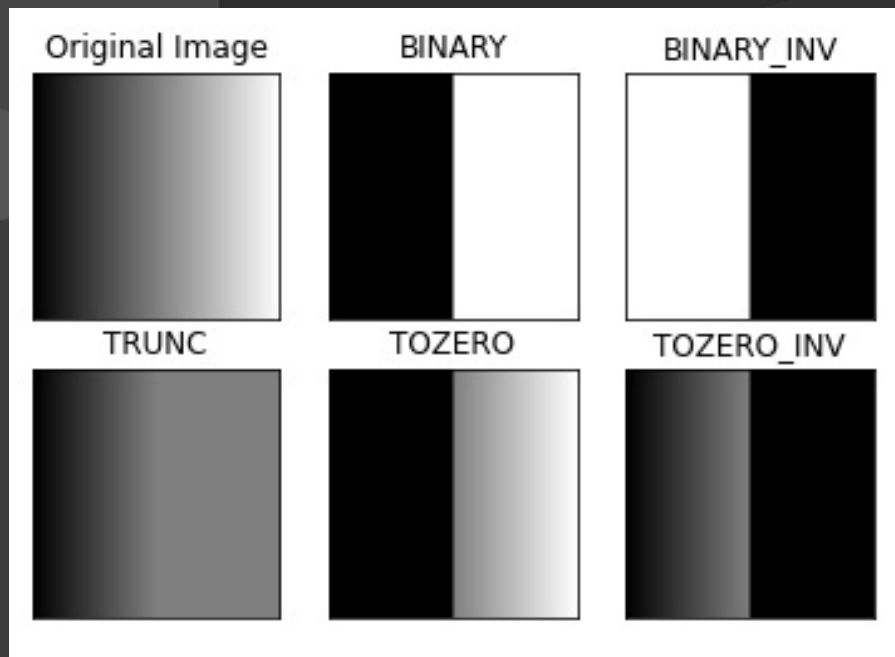


What is Image Thresholding?

- A segmentation technique that converts grayscale or RGB images to binary images.
- Divides an image into foreground and background based on pixel intensity.

Importance

- Enhances visibility and isolation of objects in images.
- Useful in applications like text extraction from distorted images.



Thresholding Techniques

1. Simple Thresholding

Definition: Compares pixel values to a specified threshold.

Types: Binary Thresholding, Inverse Thresholding, Truncate Thresholding, Threshold to Zero

```
ret,thresh1 = cv2.threshold(image, 127, 255, cv2.THRESH_BINARY)
imshow('1 Threshold Binary @ 127', thresh1)
```

2. Otsu's Thresholding

Description: Automatically finds an optimal threshold by maximizing between-class variance.

```
**cv2.adaptiveThreshold**(src, maxValue, adaptiveMethod, thresholdType, blockSize, C[, dst]) → dst
thresh = cv2.adaptiveThreshold(image, 255, cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY, 3, 5)
```

3. Adaptive Thresholding

Definition: Local thresholding technique that adjusts thresholds based on pixel neighborhoods.

Methods: Arithmetic Mean Thresholding, Gaussian Mean Thresholding

```
blur = cv2.GaussianBlur(image, (5,5), 0)
_, th3 = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

WEEK 3

In this week, we learnt about:

- *Morphological Operations*
- *Edge Detection*
- *Fourier Transform*
- *Contours*

What are Morphological Operations?

Morphological operations are non-linear image processing techniques that operate on the shape and structure of objects in a binary image.

They are mainly applied to binary images, where:

- 0 represents the background
- 1 represents the foreground (object)

CORE IDEA:

A small shape called a Structuring Element (SE) is moved over the image to examine how object pixels fit within it.

Based on this interaction, the object's shape is modified.

Why Do We Use Morphological Operations?

Morphological operations help to:

- Remove small noise
- Fill gaps and holes
- Separate connected objects
- Smooth object boundaries

These operations are especially useful as a preprocessing step before edge detection and contour extraction.

Binary Image as a Set (Simple View):

A binary image can be treated as a set of foreground pixels.

Morphological operations compare:

- The image set
- The structuring element

This set-based view explains why morphology works well for shape-based processing

EDGE DETECTION:

Edge detection is a technique in computer vision used to find object boundaries in an image.

Edges occur at locations where there is a sudden change in intensity (brightness), color, or texture.

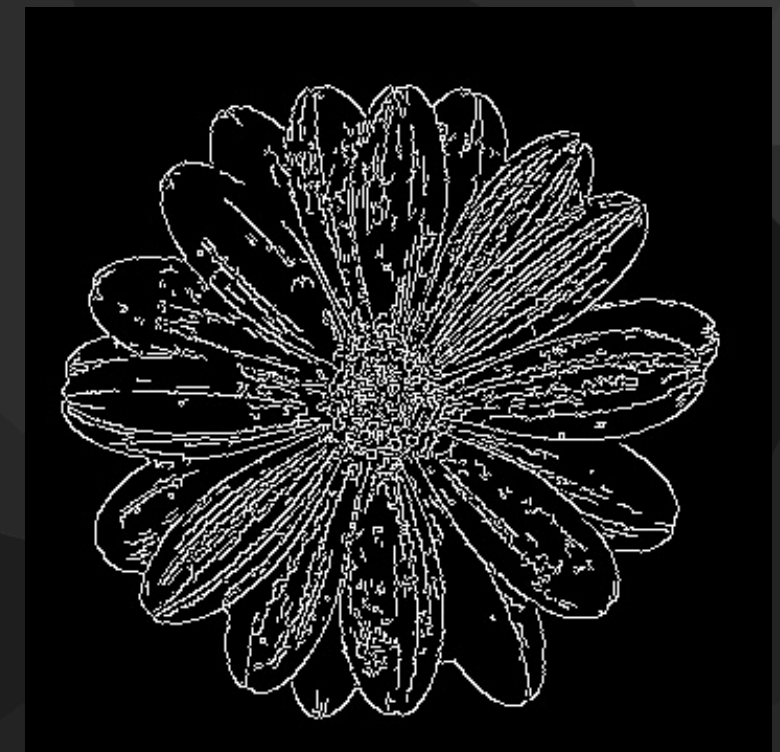
CORE IDEA:

- An image is analyzed to detect strong intensity variations
- These variations indicate the presence of edges
- Edges usually represent the outline or structure of objects

WHY EDGE DETECTION IS IMPORTANT:

Edge detection helps in:

- Object recognition
- Image segmentation
- Feature extraction
- Line and shape detection



Fourier Transform:

Fourier Transform converts a signal from its original domain into a representation in the frequency domain, revealing its spectral components.

OpenCV uses the Discrete Fourier Transform (DFT) to analyze images in the frequency domain

What FT Does in OpenCV:

Converts an image from spatial domain $(x, y) \rightarrow$ frequency domain (u, v)

- Low frequencies $\rightarrow$ smooth areas / illumination
- High frequencies $\rightarrow$ edges / fine details / noise

Contours:

Contours are curves or lines that join continuous points along the boundary of an object that have the same color or intensity.

In simple words: A contour is the outline or boundary of an object in an image.

How contours are obtained:

- Preprocess the image (convert to grayscale, apply blur, threshold, or edge detection like Canny).
- The output is a list of contours, each as a set of points (polygon approximation possible).

Contours are used for:

- Object detection
- Shape analysis
- Area & perimeter calculation
- Object counting
- Tracking

Hierarchy:

- Contours can have parent-child relationships (for holes inside objects).
- In OpenCV: `cv2.RETR_TREE`, `cv2.RETR_EXTERNAL`, etc.

WEEK 4

TOPICS WE LEARNED THIS WEEK:

- Introduction to CNN
- CNN Visualization
- CNN Futurology Visualization
- 3D Convolution Visualization
- Key Components of CNN
- Convolutional Layers

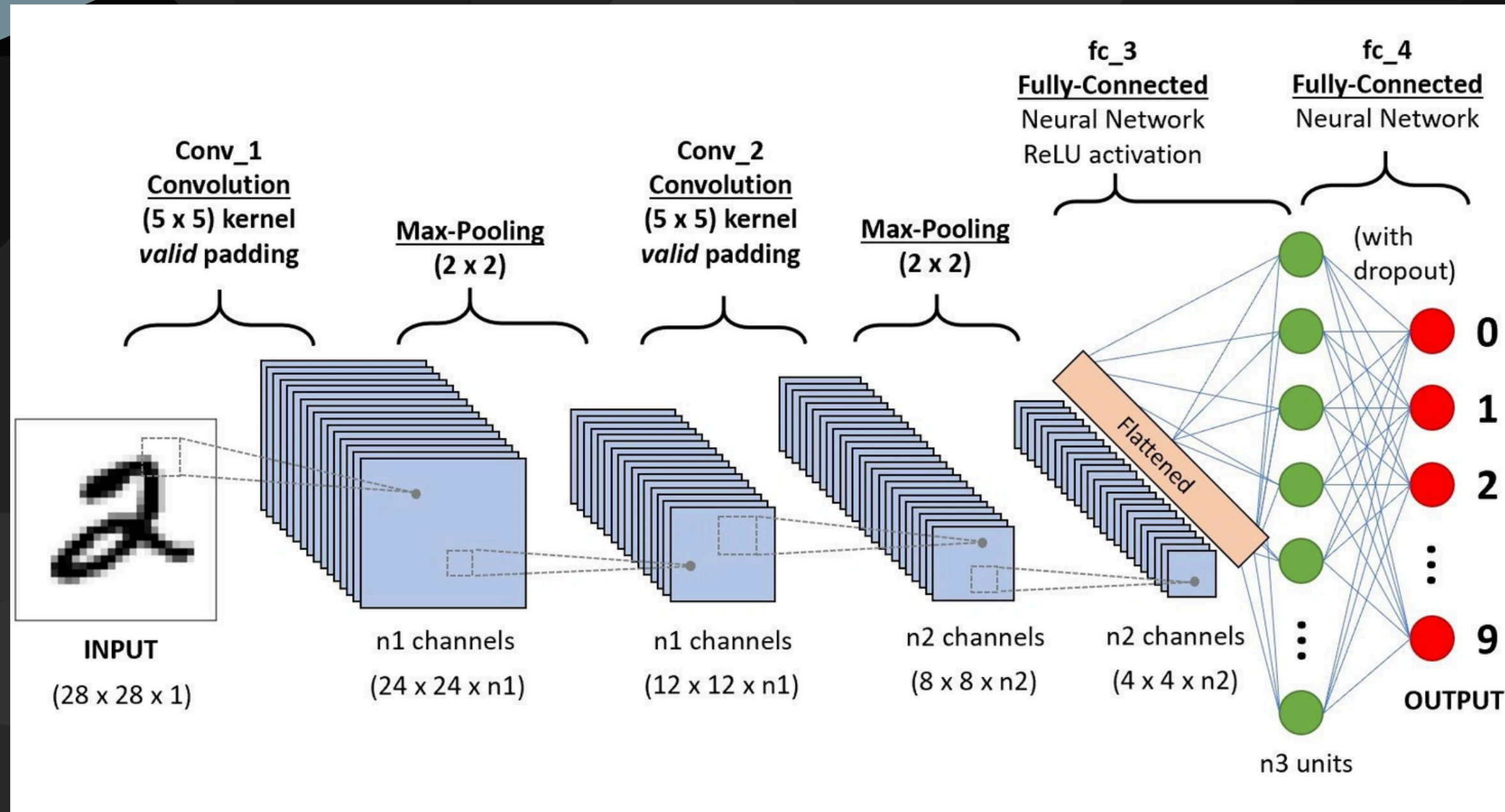
INTRODUCTION TO CNN

Definition: CNNs are specialized deep neural networks designed for grid-like data (especially images). They use trainable filters (kernels) to automatically extract features from input data.

Feature Learning: CNNs learn to detect edges, textures, shapes, etc., directly from raw data, without the need for manual feature engineering.

Applications: Widely used in computer vision (image/video recognition, classification, segmentation, etc.) and even in other domains (audio, text). They are the de facto standard for image tasks.





Minimal Preprocessing: Unlike traditional vision pipelines, CNNs require little preprocessing. They learn the optimal filters through training, making them scalable and efficient.

CNN ARCHITECTURE OVERVIEW



Layered Structure of a CNN

- A Convolutional Neural Network (CNN) processes an image in multiple stages (layers).
- Each layer performs a specific operation to gradually extract meaningful information from the image.
- The layers are arranged from low-level feature extraction to high-level decision making.



CONVOLUTION LAYERS

Learnable Filters: A convolutional layer consists of multiple small kernels (e.g. 3×3 or 5×5 matrices) that slide over the input image or feature map. Each filter performs an element-wise dot product with a local patch of the input.

Feature Maps: The result of each filter is a feature map (activation map) that highlights where that filter “detects” its learned feature (e.g. a vertical edge) in the image. Stacking all feature maps (one per filter) forms the layer’s 3D output.

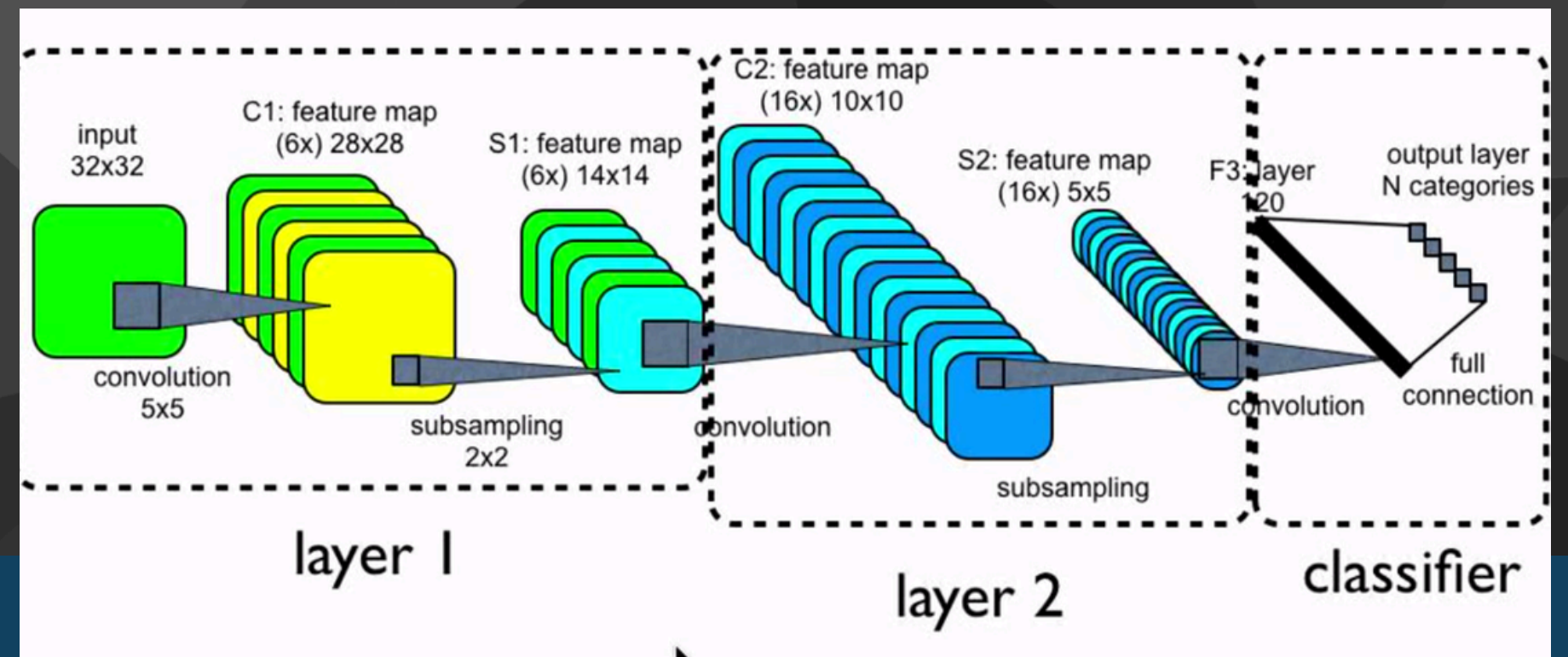
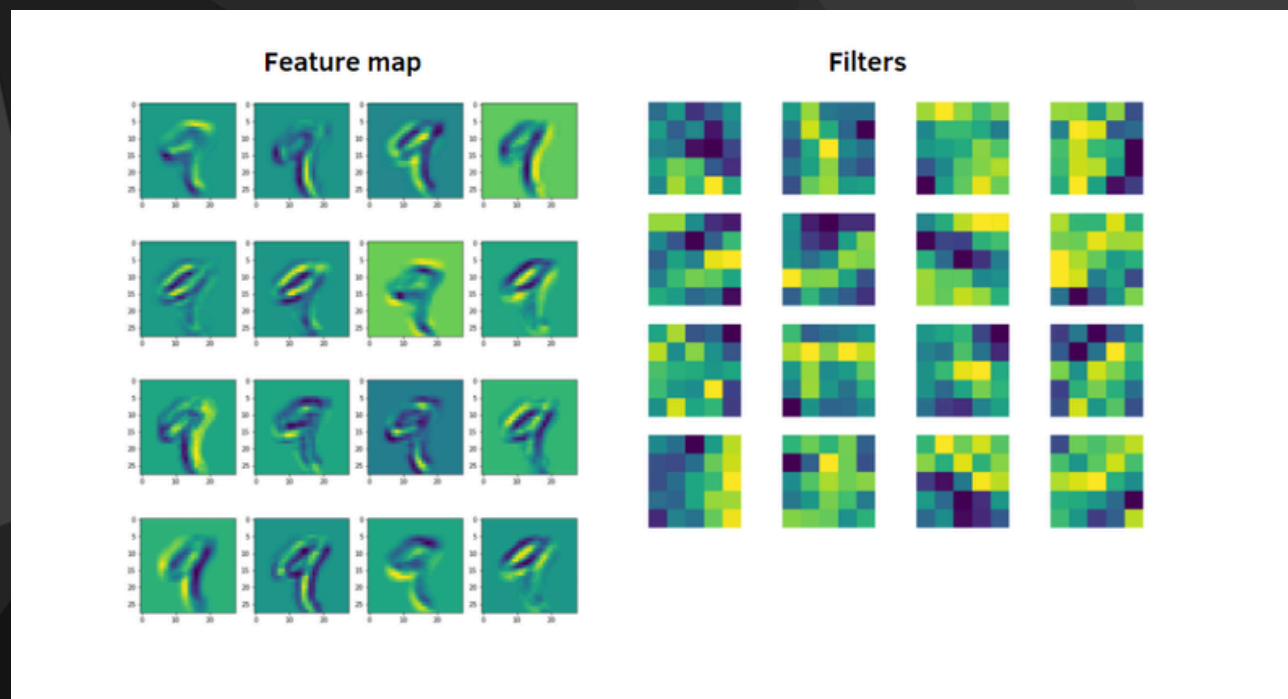
Local Connectivity & Sharing: Each neuron in the conv layer connects only to a small local region of the input (its receptive field). The same filter weights are

- shared across all spatial locations, yielding translation-equivariant detection and
- far fewer parameters.



WORKING OF A CONVOLUTION LAYER

- A filter moves across the image step by step
- At each position:
- Multiply filter values with pixel values
- Sum the results
- Store result in the feature map
- Different filters detect different features
- Multiple layers learn complex representations



POOLING LAYERS

Downsampling: Pooling layers reduce the spatial dimensions (width, height) of feature maps by summarizing local neighborhoods. This reduces computation in later layers and provides a form of translation invariance.

Common Types: The two most common pooling operations are Max Pooling and Average Pooling. In max pooling (e.g. 2×2 window), the output is the maximum value in each window; in average pooling, it's the mean.

Benefits: Pooling makes the network more robust to small shifts or distortions: if a feature moves slightly within its window, the pooled result remains the same. It also reduces overfitting by forcing a more compact representation.

Example: A 2×2 max pool with stride 2 on a 32×32 feature map yields a 16×16 output, keeping only the strongest activation in each 2×2 block.

FULLY-CONNECTED LAYERS

Fully Connected Layers are used at the final stage of a Convolutional Neural Network. In this layer, the feature maps extracted by the previous convolutional and pooling layers are flattened into a single one-dimensional vector. Every neuron in this layer is connected to every neuron in the next layer, allowing the network to combine all the learned features. Fully connected layers are responsible for making the final decision of the network, such as performing classification or regression based on the extracted features.

ACTIVATION FUNCTIONS

Activation functions introduce non-linearity into the network, allowing CNNs to learn complex patterns. The most commonly used activation function is ReLU, which helps improve training speed and model performance and is applied after convolutional and fully connected layers.

Example :

For detecting edges in an image:

Weak/no edge $\rightarrow$ value = $-2 \rightarrow$ ReLU makes it 0

Strong edge $\rightarrow$ value = 4 $\rightarrow$ ReLU keeps 4

👉 ReLU removes noise and keeps important image features, helping CNNs learn faster and better.

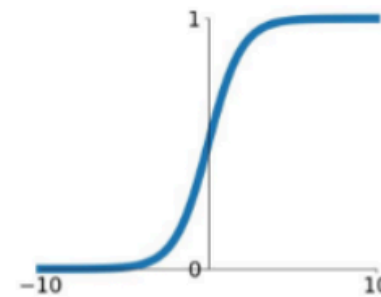
REGULARIZATION TECHNIQUES

Regularization techniques are used to prevent overfitting and improve the model's performance on unseen data. This technique uses multiple types of activation functions to give better outputs.

Activation Functions

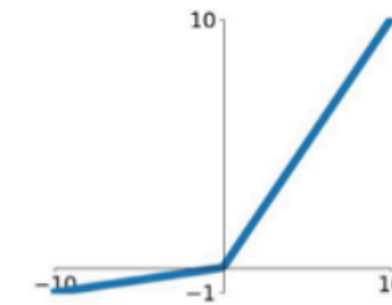
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



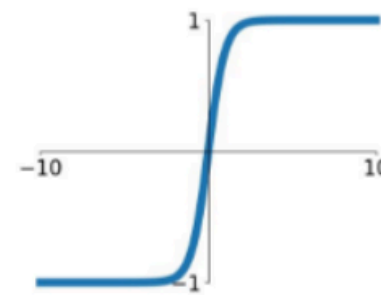
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

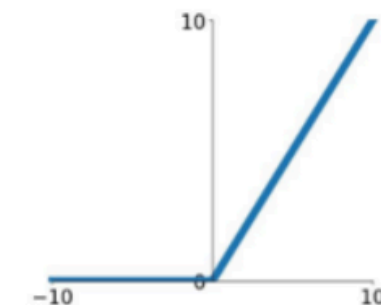


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

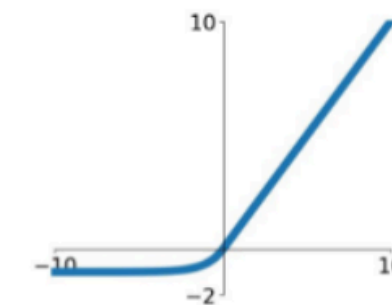
ReLU

$$\max(0, x)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

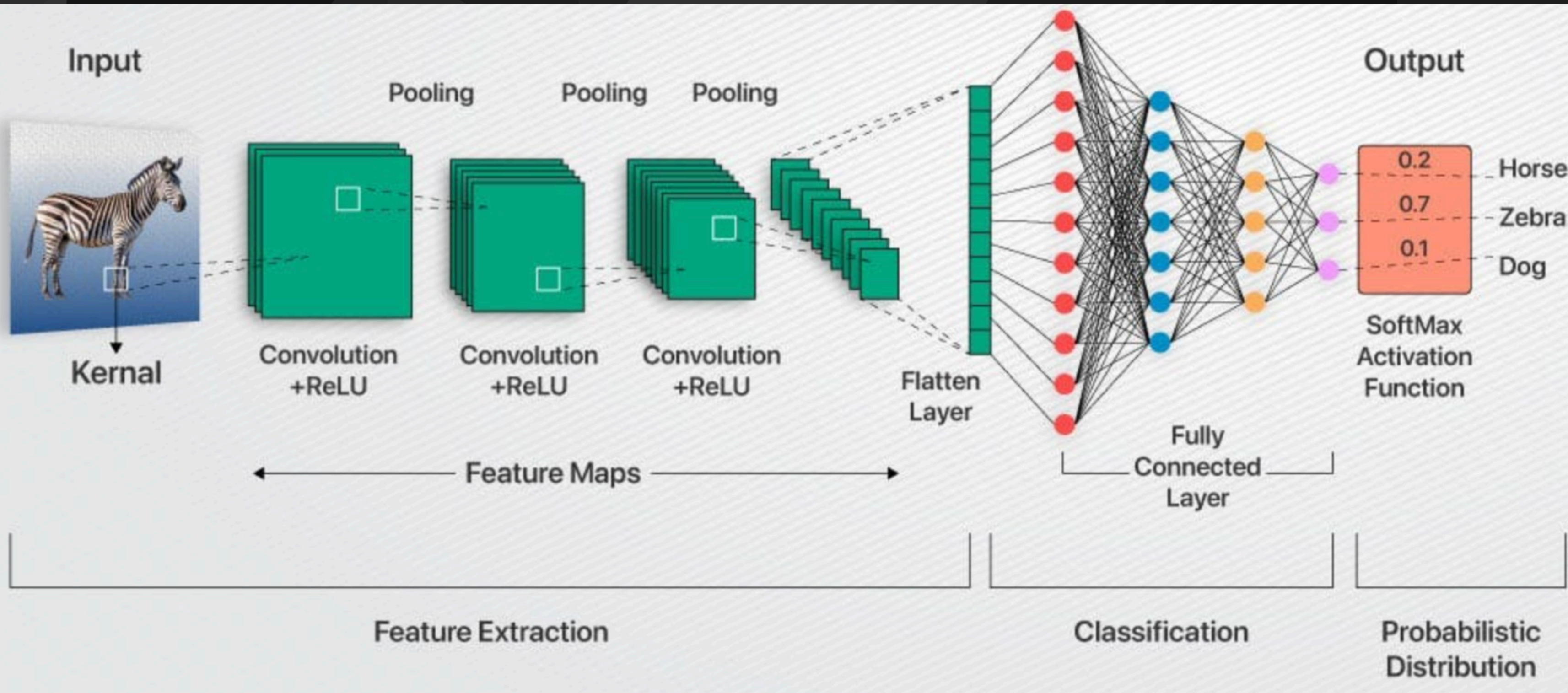


CNN VISUALIZATION

CNN visualization from scratch helps in understanding how the network processes images at each layer. It shows how filters and feature maps change through the network and how simple features gradually form complex representations.

In simple terms:

- It's a way to look inside a CNN and see what it's learning at each layer.
- Think of it like “X-ray vision” for neural networks.



Final Eval Question 2

ID Card Detected!
Width: 820 pixels
Height: 1305 pixels

Canny Edges

Boundary Contour

ROE

ROBLEMS 4

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

THANK YOU



